LAPORAN PRAKTIKUM PEMOGRAMAN BERORIENTASI OBJEK

“RELASI ANTAR OBJEK (ASOSIASI)”

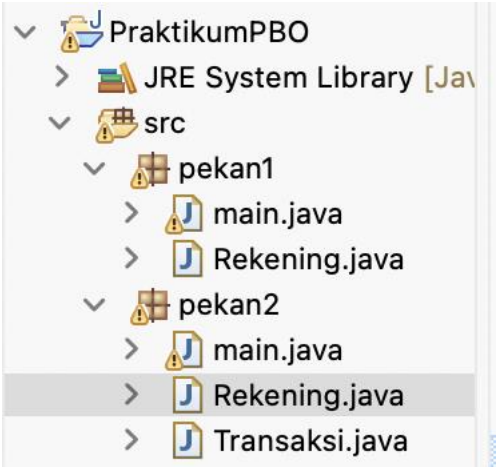
disusun Oleh:
DENNIS SHAUQI AKBAR
2511533020

Dosen Pengampu:
Nurfiah, S.ST, M.Kom



1. Langkah Kerja Praktikum

Langkah pertama yang dilakukan adalah membuka kembali aplikasi Eclipse, kemudian di dalam Java Project PraktikumPBO dibuat sebuah package baru bernama pekan2 sebagai tempat penyimpanan seluruh source code pada praktikum kedua ini. Pada praktikum ini, sistem perbankan mini yang telah dibuat pada modul sebelumnya dikembangkan agar mampu menyimpan riwayat transaksi melalui penerapan relasi asosiasi antar objek.



a. Class Transaksi

```java
package pekan2;

public class Transaksi {
    String idTransaksi;
    String jenis;
    double nominal;

    //Contructor
    public Transaksi(String id, String jenis, double nominal) {
        this.idTransaksi = id;
        this.jenis = jenis;
        this.nominal = nominal;

    }

    public void cetakDetail() {
        System.out.println("ID: " + idTransaksi + " | Jenis: " + jenis + " | Nominal: Rp" + nominal);
    }
}
```

Class Transaksi dibuat sebagai model dari satu catatan transaksi yang terjadi pada sebuah rekening. Class ini memiliki tiga atribut, yaitu idTransaksi dan jenis bertipe String, serta nominal bertipe double. Constructor Transaksi(String id, String jenis, double nominal) digunakan untuk menginisialisasi ketiga atribut tersebut pada saat objek transaksi dibuat, dengan menggunakan keyword this untuk membedakan antara atribut milik objek dan parameter yang dikirimkan. Atribut jenis hanya memiliki dua kemungkinan nilai, yaitu Kredit untuk aktivitas penyetoran dan Debit untuk aktivitas penarikan. Selanjutnya, method cetakDetail() berfungsi menampilkan satu baris informasi lengkap dari sebuah transaksi yang terdiri atas ID transaksi, jenis transaksi, dan nominalnya.

b. Relasi pada Class Rekening

```java
package pekan2;
import java.util.*;

public class Rekening {
    String nomorRekening;
    String namaPemilik;
    double saldo;

    // Implementasi Asosiasi (1-to-Many)
    ArrayList<Transaksi> riwayatTransaksi;

    public Rekening(String nomor, String nama, double saldoAwal) {
        this.nomorRekening = nomor;
        this.namaPemilik = nama;
        this.saldo = saldoAwal;

        // Wajib Menginisialisasi ArrayList di dalam constructor agar tidak NullPointerException
        this.riwayatTransaksi = new ArrayList<>();

        System.out.println("Rekening atas nama " + namaPemilik + " berhasil dibuat dengan saldo Rp" + saldo);
    }

```

Pada class Rekening, ditambahkan sebuah atribut baru berupa ArrayList<Transaksi> riwayatTransaksi yang merepresentasikan relasi has-a (asosiasi one-to-many), yaitu satu objek Rekening dapat memiliki banyak objek Transaksi di dalamnya. Atribut ini wajib diinisialisasi di dalam constructor menggunakan this.riwayatTransaksi = new ArrayList<>(); agar koleksi tersebut siap digunakan ketika objek rekening pertama kali dibuat. Tanpa inisialisasi ini, program akan mengalami NullPointerException saat mencoba menambahkan data transaksi, karena ArrayList masih bernilai null.

c. Perekaman Riwayat pada Method setorTunai

```java
    public void setorTunai(double nominal) {
        if (nominal > 0) {
            saldo += nominal;
            // Merekam riwayat (Pembuatan objek Transaksi di dalam method)
            String idTrx = "TRX-S-" + System.currentTimeMillis();
            Transaksi trxBaru = new Transaksi(idTrx, "Kredit", nominal);
            riwayatTransaksi.add(trxBaru);

            System.out.println("Setor tunai Rp" + nominal + " berhasil. Saldo saat ini: Rp" + saldo);
        } else {
            System.out.println("Gagal: Nominal setor harus lebih dari 0!");
        }
    }
```

Method setorTunai(double nominal) diperbaiki agar tidak hanya menambah saldo, tetapi juga mencatat riwayatnya. Apabila nominal yang dimasukkan lebih besar dari 0, saldo ditambah, lalu dibentuk sebuah ID transaksi dengan prefiks "TRX-S-" yang digabungkan dengan System.currentTimeMillis() agar setiap ID bersifat unik. Selanjutnya dibuat objek Transaksi baru berjenis "Kredit" yang langsung ditambahkan ke dalam riwayatTransaksi menggunakan .add(). Dengan demikian, setiap penyetoran yang berhasil akan otomatis terekam di dalam objek rekening yang bersangkutan.

2. Tugas/Latihan Praktikum

Tugas 1: Integrasi Rekam Jejak Penarikan

```
37⊖    public void tarikTunai(double nominal) {
38⊖        if (nominal < 10000) {
39
40             System.out.println("Transaksi Gagal: Minimal nominal penarikan Rp10000");
41⊖        } else if (nominal > saldo) {
42
43             System.out.println("Transaksi Gagal: Saldo tidak mencukupi. Saldo Anda: Rp" + saldo);
44⊖        } else {
45             saldo -= nominal;
46          // Merekam riwayat (Pembuatan objek Transaksi di dalam method)
47             String idTrx = "TRX-T-" + System.currentTimeMillis();
48             Transaksi trxBaru = new Transaksi(idTrx, "Debit", nominal);
49             riwayatTransaksi.add(trxBaru);
50
51             System.out.println("Tarik tunai Rp" + nominal + " berhasil. Saldo saat ini: Rp" + saldo);
52         }
53     }
```

Method tarikTunai (double nominal) dimodifikasi agar juga merekam objek Transaksi seperti pada setorTunai(). Method ini melakukan dua pengecekan secara berurutan: jika nominal kurang dari 10.000, transaksi ditolak dengan pesan minimal penarikan; jika nominal melebihi saldo, transaksi ditolak dengan pesan saldo tidak mencukupi. Perekaman transaksi diletakkan di dalam blok else, yaitu blok yang hanya dieksekusi apabila kedua syarat tersebut terpenuhi. Penempatan ini penting agar transaksi hanya tercatat ketika penarikan benar-benar berhasil, sehingga penarikan yang gagal tidak ikut masuk ke dalam riwayat. ID transaksi dibentuk menggunakan prefiks "TRX-T-" dan jenis transaksinya dicatat sebagai "Debit" sesuai ketentuan pada modul.

Tugas 2: Fitur Cetak Mutasi Rekening

a. Class Rekening

```
55⊖    public void cetakMutasi() {
56⊖        if (riwayatTransaksi.isEmpty()) {
57             System.out.println("Belum ada transaksi pada rekening ini");
58⊖        } else {
59⊖            for (Transaksi t : riwayatTransaksi) {
60                 t.cetakDetail();
61
62             }
63
64         }
65     }
```

Method cetakMutasi() dibuat di dalam class Rekening untuk menampilkan seluruh riwayat transaksi yang dimiliki sebuah rekening. Method ini terlebih dahulu memeriksa apakah riwayatTransaksi masih kosong menggunakan .isEmpty(). Jika kosong, program menampilkan pesan bahwa belum ada transaksi pada rekening tersebut. Jika tidak kosong, dilakukan perulangan for-each yang menelusuri setiap objek Transaksi di dalam koleksi riwayatTransaksi, kemudian

memanggil method cetakDetail() milik masing-masing objek tersebut. Di sinilah letak interaksi antar objek terlihat jelas: objek Rekening tidak mencetak data transaksi secara langsung, melainkan mendelegasikan tugas tersebut kepada objek Transaksi yang dimilikinya.

b. Class Main

```
93                  case 6:
94                  if (akunAktif == null) {
95                      System.out.println("Error: Anda belum membuka rekening!");
96                  } else {
97                      akunAktif.cetakMutasi();
98
99              }
100                 break;
```

Pada class Main, ditambahkan menu baru yaitu "6. Cetak Mutasi (Riwayat)". Pada case 6, program terlebih dahulu memeriksa apakah akunAktif masih bernilai null. Jika belum ada rekening yang aktif, ditampilkan pesan error. Jika sudah ada, program memanggil method cetakMutasi() milik akunAktif sehingga seluruh riwayat transaksi pada rekening tersebut ditampilkan ke konsol.

Output:

Pengujian pertama dilakukan pada rekening yang baru dibuat dan belum pernah melakukan transaksi apa pun:

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
0. Keluar
Pilih menu: 6
Belum ada transaksi pada rekening ini
```

Selanjutnya dilakukan dua kali penyetoran, masing-masing sebesar Rp35.000 dan Rp15.000:

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
0. Keluar
Pilih menu: 2
Masukkan nominal setor: 35000
Setor tunai Rp35000.0 berhasil. Saldo saat ini: Rp85000.0

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
0. Keluar
Pilih menu: 2
Masukkan nominal setor: 15000
Setor tunai Rp15000.0 berhasil. Saldo saat ini: Rp100000.0
```

Kemudian dilakukan satu kali penarikan tunai sebesar Rp50.000, dan berhasil karena memenuhi syarat minimal penarikan serta tidak melebihi saldo yang tersedia:

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
0. Keluar
Pilih menu: 3
Masukkan nominal tarik: 50000
Tarik tunai Rp50000.0 berhasil. Saldo saat ini: Rp50000.0
```

Setelah ketiga transaksi tersebut dilakukan, menu Cetak Mutasi dipilih kembali untuk melihat seluruh riwayat transaksi yang telah tercatat pada rekening tersebut:

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
0. Keluar
Pilih menu: 6
ID: TRX-S-1789835859356 | Jenis: Kredit | Nominal: Rp35000.0
ID: TRX-S-1789835864914 | Jenis: Kredit | Nominal: Rp15000.0
ID: TRX-T-1789835916890 | Jenis: Debit | Nominal: Rp50000.0
```

Hasil pengujian menunjukkan bahwa setiap transaksi setor dan tarik yang berhasil dilakukan secara otomatis membentuk objek Transaksi baru dan menyimpannya ke dalam riwayatTransaksi milik rekening yang bersangkutan. Kedua transaksi setor tercatat sebagai jenis Kredit dengan prefiks ID TRX-S-, sedangkan transaksi tarik tercatat sebagai jenis Debit dengan prefiks ID TRX-T-, sesuai dengan ketentuan yang diminta pada modul. Hal ini membuktikan bahwa relasi asosiasi one-to-many antara objek Rekening dan Transaksi telah berhasil diimplementasikan dengan baik.